

Projeto Final da Disciplina de Aprendizagem de Máquina

Prof. Leandro Almeida

2026-1

1 Objetivo Geral

Este projeto visa proporcionar aos alunos uma experiência prática aprofundada em modelos de classificação supervisionada recentes e avançados, incluindo a análise detalhada do funcionamento, a comparação sistemática e a avaliação rigorosa dos modelos selecionados sobre dados tabulares do benchmark *TabArena-v0.1* (NeurIPS 2025).

2 Descrição do Trabalho

Cada grupo terá atribuído um modelo recente de classificação supervisionada para estudo e implementação. Após a análise do modelo sugerido, o grupo pode apresentar outras opções de modelos recentes com a devida justificativa. Uma única solicitação de mudança por grupo será aceita para deferimento ou não. O projeto final da disciplina compreende duas etapas: o entendimento teórico do modelo atribuído e a realização de experimentos comparativos. Para ambas as etapas devem ser apresentadas as seguintes informações:

Tempo de apresentação: cada grupo terá um tempo total de **20 minutos** para apresentar o trabalho, organizado em dois blocos contíguos de 10 minutos: o primeiro dedicado à **Etapa 1** (apresentação teórica do modelo atribuído) e o segundo dedicado à **Etapa 2** (resultados experimentais, comparações, testes estatísticos e conclusões). O tempo será controlado pelo professor.

2.1 Etapa 1 - Apresentação Teórica do Modelo

Nos primeiros 10 minutos do seminário, cada grupo apresentará o modelo atribuído. A apresentação deve contemplar:

- Motivação e contexto histórico do modelo;
- Funcionamento detalhado do modelo (arquitetura, hiperparâmetros principais, peculiaridades e funcionamento interno);
- Forma de aprendizado e representação dos padrões nos dados;
- Aplicações práticas e limitações.

2.2 Etapa 2 - Experimentos Comparativos

Nesta etapa, cada grupo realizará experimentos comparando o seu modelo com mais três modelos amplamente utilizados (*LightGBM*, *CatBoost*, *XGBoost*) e com duas configurações da abordagem *AutoML* (*AutoGluon 1.4* no preset *default* e no preset *extreme* de 4 horas, este último incluindo internamente foundation models como TabPFN v2).

Bases de Dados: Serão utilizados **30 datasets no total**, selecionados do benchmark **TabArena-v0.1** (NeurIPS 2025), curado manualmente para garantir relevância prática, estrutura tabular IID e licenciamento adequado. A seleção deve ser estratificada por regime de tamanho da seguinte forma:

- **10 datasets de regime pequeno** (n menor que 1.000 amostras);
- **10 datasets de regime médio** (entre 1.000 e 10.000 amostras);
- **10 datasets de regime grande** (acima de 10.000 amostras).

Recomenda-se que cada regime contenha pelo menos um problema de classificação binária e um de classificação multiclasse, sempre que a oferta do TabArena permitir. A relação final dos datasets escolhidos deve constar no relatório como tabela contendo: nome do dataset, identificador da *task* no OpenML, número de amostras (n), número de features, número de classes e regime atribuído.

Framework de Comparação: Deverá ser utilizado o protocolo experimental de comparação de múltiplos classificadores proposto por **Demšar (2006)** (teste de Friedman, post-hoc Nemenyi/Holm e diagrama de diferença crítica via biblioteca *autorank*), complementado pela análise Bayesiana de **Benavoli, Corani, Demšar e Zaffalon (JMLR, 2017)** (Bayesian signed-rank e Bayesian hierarchical t-test com região de equivalência prática, ROPE, via biblioteca *baycomp*).

Procedimentos:

1. Dividir cada base de dados em conjuntos de treinamento e teste (70% treino, 30% teste, com seed fixa documentada);
2. Realizar busca por hiperparâmetros para todos os modelos considerando validação cruzada no conjunto de treino (sugere-se *Optuna*);
3. Avaliar os modelos usando as seguintes métricas:
 - Mean AUC OVO (One-vs-One);
 - Mean Accuracy (ACC);
 - G-Mean;
 - Mean Cross-Entropy (CE);
 - Tempo médio em segundos / custo computacional (tune + train + predict).
4. Apresentar os resultados finais de treino e teste usando a melhor configuração obtida para cada modelo deste projeto;
5. Realizar análise por regime, segmentando os resultados por (i) tamanho do dataset (n menor que 1k, 1k a 10k, acima de 10k), (ii) número de classes (binário vs. multiclasse), (iii) proporção entre features numéricas e categóricas, e (iv) presença de valores ausentes;

6. Documentar todo o pipeline em ambiente reproduzível, contendo `pyproject.toml` (gerenciado por `uv` ou `poetry`) com versões fixadas, seed fixa em todas as etapas e, opcionalmente, `Dockerfile` para containerização;
7. Produzir um *model card* detalhado do modelo principal do grupo, seguindo o template em `model_cards/TEMPLATE.md` do repositório (onze seções inspiradas em Mitchell et al. (2019) com extensões da disciplina: detalhes do modelo, uso pretendido, fatores observados nos quatro regimes, métricas com intervalo de confiança de 95%, dados de avaliação, dados de treino e pré-treino, análise quantitativa contra os 14 demais sistemas, considerações éticas, avisos e recomendações, reprodutibilidade e referências).

Nos 10 minutos finais do seminário, o grupo apresentará todo o fluxo de trabalho da Etapa 2, incluindo comparações entre modelos, testes estatísticos (clássicos e Bayesianos) e conclusões acerca do desempenho.

3 Modelos Selecionados

A seguir os modelos recentes (2023 a 2026) a serem atribuídos aos grupos. Todos possuem repositórios públicos ativos, instalação reproduzível via `pip` e licenças permissivas para uso acadêmico:

1. **TabPFN-2.5 (Tabular Prior-Data Fitted Network, versão 2.5)**
<https://github.com/PriorLabs/TabPFN>
2. **TabICL v2 (Tabular Foundation Model for In-Context Learning)**
<https://github.com/soda-inria/tabicl>
3. **TabM (Parameter-Efficient Ensembling, ICLR 2025)**
<https://github.com/yandex-research/tabm>
4. **ModernNCA (Modern Neighbor Component Analysis, ICLR 2025)**
<https://github.com/LAMDA-Tabular/TALENT>
5. **RealMLP (NeurIPS 2024)**
<https://github.com/dholzmueller/pytabkit>
6. **xRFM (Recursive Feature Machine, 2025)**
<https://github.com/dmbeaglehole/xRFM>
7. **Mambular (Tabular Mamba/SSM, 2024)**
<https://github.com/OpenTabular/DeepTab>
8. **FT-Transformer (Feature Tokenizer + Transformer)**
<https://github.com/dholzmueller/pytabkit>
9. **EBM (Explainable Boosting Machine)**
<https://github.com/interpretml/interpret>
10. **Trompt (Tabular Prompt, ICML 2023)**
<https://github.com/pyg-team/pytorch-frame>

4 Infraestrutura Recomendada

Para garantir reprodutibilidade e reduzir o tempo de configuração inicial dos grupos, recomendam-se os seguintes pacotes (todos disponíveis via `pip`):

- **pytabkit**: API *scikit-learn* unificada para LightGBM, CatBoost, XGBoost, RealMLP, TabM, FT-Transformer e variantes pré-configuradas (defaults meta-aprendidos);
- **deeptab**: implementações *scikit-learn* de Mambular, FT-Transformer, TabTransformer e ResNets tabulares;
- **interpret**: implementação oficial do EBM (Microsoft Research);
- **pytorch-frame**: biblioteca de *tabular deep learning* mantida pela equipe do PyTorch Geometric, com exemplo oficial de Tromp;
- **LAMDA-Tabular/TALENT**: toolkit que reúne mais de 35 métodos de *tabular deep learning*, incluindo ModernNCA;
- **tabpfn**, **tabicl**, **xrfm**: pacotes oficiais dos respectivos modelos;
- **autogluon.tabular**: AutoML de referência (preset *default* e *extreme*);
- **optuna**: busca de hiperparâmetros;
- **autorank**, **baycomp**: testes estatísticos clássicos (Friedman, Nemenyi, CD diagram) e Bayesianos (signed-rank, hierarchical t-test, ROPE).

5 Entrega e Avaliação

Cada grupo deve entregar:

- Slides utilizados na apresentação sobre o modelo;
- Slides com resultados detalhados do experimento, incluindo gráficos e tabelas das métricas avaliadas, diagrama de diferença crítica e análise Bayesiana com ROPE;
- Relatório descrevendo o trabalho realizado e os resultados alcançados, contendo a tabela detalhada dos 30 datasets selecionados (nome, *task* OpenML, *n*, número de features, número de classes e regime) e a análise por regime descrita nos Procedimentos;
- Código utilizado para realizar os experimentos, em ambiente reproduzível (`pyproject.toml` com versões fixadas e, opcionalmente, `Dockerfile`);
- Estrutura sugerida do relatório em `entregaveis/relatorio-template.pdf` do repositório;
- Estrutura sugerida dos slides em `entregaveis/slides-template.pdf` do repositório (cerca de 21 frames, dois blocos contíguos de 10 minutos);
- Rubrica de avaliação detalhada em `entregaveis/rubrica.pdf` do repositório, decompondo os pesos 40%, 50% e 10% deste documento em critérios específicos;
- *Model card* detalhado do modelo principal do grupo, conforme `model_cards/TEMPLATE.md` do repositório.

A avaliação será baseada em:

- Clareza e profundidade na explicação do modelo (40%);
- Qualidade e rigor nos experimentos realizados (50%);
- Qualidade da apresentação (10%).

6 Repositório-Template

Os grupos receberão como anexo deste projeto um repositório-template oficial contendo a estrutura de código de referência: módulos para carregamento dos datasets do TabArena-v0.1, implementação dos baselines via `pytabkit`, integração com AutoGluon, pipeline padronizado de *split*, *tuning*, *evaluation* e análise estatística (`autorank` e `baycomp`), notebooks de demonstração e *smoke test* de validação. O uso do template é fortemente recomendado para reduzir o esforço de configuração inicial e padronizar a comparação entre os grupos. O arquivo `README.md` do template apresenta uma seção *Status do template* indicando o que já está implementado e o que cabe a cada grupo preencher; o documento `docs/INFRAESTRUTURA.md` traz um tutorial detalhado sobre as ferramentas utilizadas.

A Anexo: Glossário Técnico

Este anexo apresenta uma referência conceitual breve sobre as ferramentas e termos técnicos utilizados no projeto. Para tutoriais passo a passo, com comandos e *troubleshooting*, consulte o arquivo `docs/INFRAESTRUTURA.md` no repositório-template.

Ambiente virtual (*virtual environment* ou *venv*). Diretório isolado que contém uma instalação Python independente, com suas próprias bibliotecas. Permite que cada projeto tenha versões específicas de pacotes sem conflitar com outros projetos ou com o Python global do sistema. No Python moderno, é criado com `python -m venv .venv` e ativado conforme o sistema operacional.

pip. Gerenciador de pacotes oficial do Python. Instala bibliotecas a partir do repositório PyPI. Comando típico: `pip install nome-do-pacote`. É a opção mais tradicional, porém mais lenta para projetos com muitas dependências.

uv. Gerenciador de pacotes e ambientes desenvolvido pela Astral (2024-2025), escrito em Rust. Substitui simultaneamente `pip`, `venv` e `pip-tools`, sendo entre 10 e 100 vezes mais rápido. Lê o `pyproject.toml` e cria o ambiente automaticamente com `uv sync`. Recomendado neste projeto.

poetry. Alternativa madura ao `pip`, popular desde 2018. Também usa `pyproject.toml` e gerencia ambiente, dependências e publicação de pacotes. Mais lento que `uv`, porém com ampla base instalada.

pyproject.toml. Arquivo padrão (PEP 621) que descreve metadados e dependências de um projeto Python. Substitui o antigo `setup.py`. É lido automaticamente por `pip`, `uv`, `poetry` e ferramentas como `pytest` e `ruff`.

Dockerfile e contêineres. Receita textual que descreve como construir uma imagem Docker, ou seja, um sistema operacional mínimo já configurado com Python, dependências e o código do projeto. Permite reproduzir o ambiente em qualquer máquina (Windows, Linux, macOS, servidor de pesquisa). Construído com `docker build` e executado com `docker run`. Opcional neste projeto, mas recomendado para reprodutibilidade plena.

Seed (semente aleatória). Número inteiro fixo passado a geradores pseudoaleatórios para garantir que sorteios (split treino/teste, inicialização de pesos, amostragem de hiperparâmetros) sejam reproduzíveis. Sem *seed* fixa, dois experimentos no mesmo código podem produzir resultados levemente diferentes.

Cross-validation (validação cruzada). Técnica para estimar desempenho de modelos sem usar o conjunto de teste final. Os dados de treino são divididos em k partes (*folds*); o modelo é treinado em $k-1$ delas e validado na restante, repetindo k vezes. A métrica média é mais robusta que um único *hold-out*.

Jupyter Notebook. Documento interativo (`.ipynb`) que mistura código Python, texto formatado e resultados (gráficos, tabelas) em células executáveis. Útil para EDA, prototipação e relatórios. Aberto via `jupyter notebook` ou `jupyter lab`.

pytest. *Framework* de testes automatizados em Python. Verifica que funções produzem o resultado esperado para entradas conhecidas. No projeto, o *smoke test* (`tests/test_pipeline.py`) confirma que cada modelo executa *end-to-end* antes de o grupo iniciar o experimento completo.

Optuna. Biblioteca para busca de hiperparâmetros baseada em otimização Bayesiana (algoritmo TPE) e em *pruning* de *trials* ruins. Mais eficiente que *grid search* aleatório para espaços de busca grandes.

OpenML. Plataforma colaborativa que hospeda milhares de *datasets* tabulares com identificadores únicos (*task IDs*) e metadados padronizados. O TabArena-v0.1 referencia datasets do OpenML; o pacote Python `openml` permite carregá-los diretamente em `pandas.DataFrame`.

Model card. Documento curto e padronizado que descreve um modelo de aprendizado de máquina: arquitetura, dados de treinamento, métricas, limitações conhecidas e considerações éticas. Inspirado em Mitchell et al. (2019), virou prática recomendada para qualquer modelo destinado a uso ou divulgação.

ROPE (Region Of Practical Equivalence). Intervalo em torno do zero dentro do qual a diferença entre dois modelos é considerada irrelevante na prática. Em testes Bayesianos (Benavoli et al., 2017), permite afirmar não apenas que dois modelos diferem, mas também que são *praticamente equivalentes*, algo que o teste de hipótese clássico não consegue.